
CS547: Advanced Topics in Software Engineering

Assessed Assignment

for

Assignment 1 Part 2

Prepared by Ali Nasir & Hamish Hartley

Group = CS547Assignment1Part2Group38

Submitted to: Marc Roper

Table of Contents

Contents

1. Implementation of the Genetic Algorithm (GA)	1
1.1 Representation	1
1.2 Fitness Function	1
1.3 Mutation	1
1.4 Crossover	1
1.5 Selection	1
1.6 Implementation	1
1.7 Results and Evaluation: Small Problem	2
1.8 Results and Evaluation: Big Problem	3

1. Implementation of the Genetic Algorithm (GA)

1.1 Representation

A subset of the test case (t1, t2, t3, etc.) is used to represent each individual in the test set. In the test instance, 1 denotes inclusion and 0 denotes exclusion. It is appropriate to use a binary representation since every test case is either chosen or not. This enables the GA to directly choose a subset of tests instead of ranking them.

1.2 Fitness Function

The fitness function used in this project is based:

1. **Maximizing Fault Coverage:** The fitness function calculates the total fault coverage by summing the number of unique faults detected by the selected tests. Since the goal is to maximize fault coverage, it is represented as a positive weight
2. **Minimizing Execution Time:** The fitness function also calculates the total execution time of the selected tests by summing their execution times. Since this should be minimized, it is represented as a negative weight.

1.3 Mutation

Bit flip mutation is used, which introduces randomness and helps explore the solution space more effectively. This allows the GA to explore new combinations by flipping selected tests on or off.

1.4 Crossover

Uniform crossover is used with a probability of 0.5 (50%) as each bit is chosen from either parent with equal probability. This operator helps create diverse offspring.

1.5 Selection

The DEAP's NSGA-II as a selection mechanism is used for multi-objective optimization. It maintains a Pareto front of non-dominated solutions, balancing between fault coverage and execution time.

1.6 Implementation

The GA was configured using DEAP framework with the following parameters:

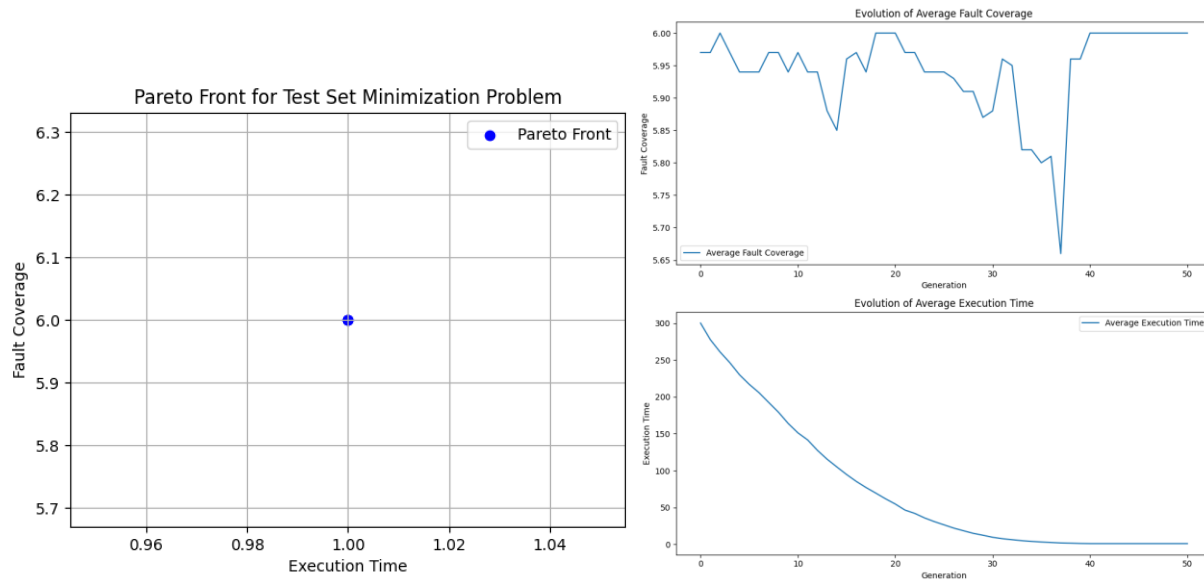
- Number of test: 100
- Population Size: 100
- Number of Generations: 50
- Crossover Probability: 0.7
- Mutation Probability: 0.2

Each generation produces a new population through crossover, mutation, and selection by NSGA-II to maintain a Pareto front of non-dominated solutions.

1.7 Results and Evaluation: Small Dataset

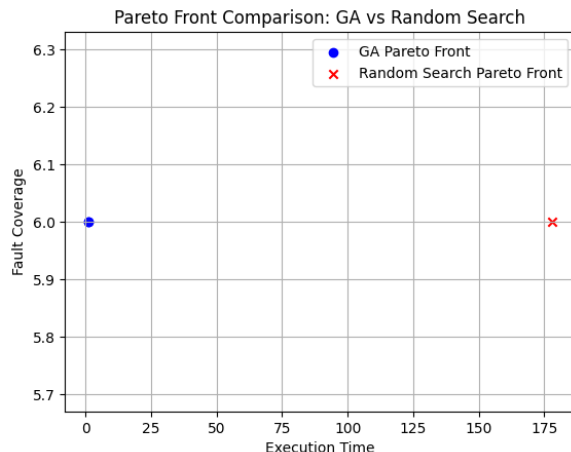
1.7.1 Genetic Algorithm

Observing the Pareto front for the small dataset reveals only one optimal solution, this is surprising however not totally unexpected due to the small dataset size which restricts the set of possible solutions. Furthermore, when studying the evolution of average fault coverage it is clear that the optimization evolution is not linear. After approximately 40 generations the solution converges on the optimal solution. This is contrasted with the steady downwards curve of evolution of average execution time.



1.7.2 Random Search

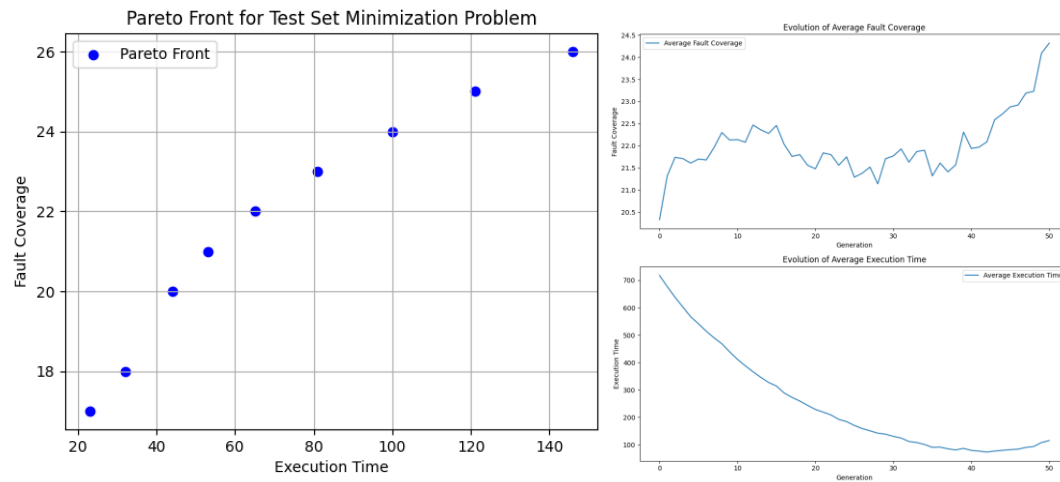
The Pareto front for both the Genetic Algorithm and random search is equivalent on the y-axis each providing a solution with a fault coverage of 6.0, the GA's solution has a substantially smaller execution time thus providing more optimized solution. This is due to the NSGA-II algorithm, which can successfully optimize a multi-objective problem.



1.8 Results and Evaluation: Big Dataset

1.8.1 Genetic Algorithm

There are 9 non-dominated solutions for the larger dataset, forming a clear Pareto front. This again is to be expected due to the larger dataset size allowing for a larger set of possible solutions to the problem. This is further echoed in the evolution plot for the average fault coverage, which has a much clearer upward trend across each generation, it is still not a linear increase however. Similarly to the smaller dataset the evolution of average execution time steadily decreases across each generation.



1.8.2 Random Search

Similarly to the smaller dataset the Pareto front for both the random search and Genetic Algorithm have largely similar fault coverage values, however their execution times differ greatly. With the Genetic Algorithm having far superior generated individual solutions which compose its Pareto front.

